

Detecting Collaboration Antipatterns in Perforce Server Logs

with Stream Metrics and Complex Event Processing

Hanifah ALPHA-BODA
Laboratoire d'Informatique Formelle
Université du Québec à Chicoutimi
Chicoutimi, CANADA
haboda@etu.uqac.ca

Yannick FRANCILETTE
Laboratoire d'Informatique Formelle
Université du Québec à Chicoutimi
Chicoutimi, CANADA
yfrancil@uqac.ca

Sylvain HALLE
Laboratoire d'Informatique Formelle
Université du Québec à Chicoutimi
Chicoutimi, CANADA
shalle@uqac.ca

Résumé—Les serveurs de gestion de versions enregistrent silencieusement chaque action d'une équipe de développement, mais ces journaux sont rarement exploités pour repérer une collaboration dysfonctionnelle. En production de jeux, Perforce journalise une activité intense sur des assets binaires Unreal (.uasset, .umap) : les signaux de processus proviennent des métadonnées de log (commandes, chemins, durées), et non de diffs textuels. Nous présentons PERFORCELOG, un outil qui détecte cinq antipatterns de collaboration—mauvais comportements observables, sur un sous-ensemble des 168 antipatterns ReliSA—directement à partir des logs serveur Perforce. L'outil assemble des blocs d'événements et suit une conception en deux phases. La première reproduit hors ligne les quatre types d'instruments OpenMetrics popularisés par Prometheus (Counter, Gauge, Histogram, Summary) [2], [3] afin de caractériser l'activité globale du dépôt. La seconde ajoute du traitement d'événements complexes (CEP) avec BEEPBEAP [5] pour reconnaître des motifs sensibles à l'ordre qu'un compteur sans mémoire ne peut exprimer, comme un fichier modifié mais jamais soumis. Sur un journal de 1,7 Go issu d'un cours de développement de jeux multi-équipes (Unreal 5.6), l'outil extrait 16 262 événements uniques et traite tout le flux en environ quatorze secondes. Il signale 217 séquences *modification-puis-annulation* et 250 fichiers abandonnés, avec un pic de 60 utilisateurs distincts par heure ; ce dernier nombre est confirmé indépendamment par la valeur résiduelle de la jauge de fichiers ouverts, ce qui fournit une vérification de cohérence interne entre les deux phases. Nous décrivons les définitions des antipatterns, l'architecture en flux et les résultats empiriques, puis discutons des limites d'une approche purement fondée sur les journaux.

Index Terms—traitement d'événements complexes, fouille de dépôts logiciels, antipatterns de collaboration, gestion de versions, Perforce, métriques

I. INTRODUCTION

Les projets de jeux modernes sont réalisés par des équipes qui partagent un même dépôt de gestion de versions. En contexte pédagogique, ce partage est amplifié : des dizaines d'étudiants soumettent sur le même serveur Perforce sous la pression des échéances, et la collaboration qui en résulte est inégale. Le serveur Perforce journalise fidèlement chaque `edit`, `submit`, `sync` et `revert`, mais le journal brut—plusieurs gigaoctets de texte semi-structuré—est en pratique

illisible à la main et n'est presque jamais utilisé comme source de rétroaction.

L'activité VCS y est intense et hétérogène, surtout sur des **fichiers binaires** Unreal (.uasset, .umap) non fusionnables comme du code source : l'analyse repose sur les séquences `edit/sync/submit/revert`, les durées et les métadonnées de log, et non sur des diffs textuels. Les logs `p4d` restent ainsi peu exploités pour la santé du *processus* collaboratif.

Cet papier décrit PERFORCELOG, un outil qui transforme un tel journal en signaux exploitables d'*antipatterns de collaboration* : comportements d'équipe récurrents et indésirables, comme un membre qui modifie de nombreux fichiers mais les partage rarement, ou un fichier ouvert puis abandonné. ReliSA recense **168 antipatterns** [1] ; dans notre travail, un antipattern est un **mauvais comportement** détectable dans les traces `user-*`. La majorité des antipatterns RELISA (réunions, management, exigences) n'apparaît pas dans `p4d` ; nous en opérationnalisons **cinq**, listés au tableau I.

TABLE I
CINQ ANTIPATTERNS OPÉRATIONNALISÉS (5 FICHES SUR 168 RELISA).

Notre détection	Fiche ReliSA
Rush de submits	Collective_Procrastination
Faible activité (Warm Bodies)	Warm_Bodies
Ratio edit/submit élevé (Lone Wolf)	Lone-Wolf
Checkout sans submit (apathie)	Bystander_Apathy
Edit puis revert	Cascading_Branches (approx.)

Notre contribution n'est pas une nouvelle théorie de détection mais une contribution d'ingénierie : un pipeline en flux qui passe à l'échelle de journaux de plusieurs gigaoctets et qui combine deux vues complémentaires du même flux d'événements. La conception est volontairement *étagée* : nous reproduisons d'abord ce que calculerait une chaîne de supervision classique—métriques agrégées Prometheus/OpenMetrics [2], [3]—avant d'ajouter le traitement d'événements complexes [4] avec BEEPBEAP [5] pour capturer l'ordre des événements. Un compteur répond « combien ? » sans mémoire ; une requête CEP répond « dans quel ordre sur ce fichier ? » en maintenant

un état. Plusieurs antipatterns ne s'expriment que sous cette seconde forme.

Contributions. (1) Parseur streaming par **blocs d'événement** (commande + completed + métriques --, liés par pid); (2) métriques Prometheus/OpenMetrics hors ligne via BEEPBEEP; (3) CEP à état pour motifs temporels au-delà des agrégats.

II. EASE OF USE

Fouille de dépôts logiciels. De nombreux travaux extraient de l'information de processus et sociale des systèmes de gestion de versions et de suivi de problèmes [6]. La plupart visent Git et opèrent par lots sur le graphe de commits. Les journaux serveur Perforce sont comparativement peu étudiés et ont une granularité différente : ils enregistrent des opérations clientes au niveau du fichier (ouverture en modification, soumission, synchronisation, annulation) plutôt que seulement les changements validés, ce qui rend observable un comportement sensible à l'ordre tel que « ouvert mais jamais soumis ».

Antipatterns de collaboration. La notion d'antipattern, une solution courante mais finalement contre-productive a été popularisée pour l'architecture logicielle et les projets [7]. Au niveau de l'équipe, des travaux récents sur les *community smells* détectent des dysfonctionnements organisationnels à partir des traces de collaboration [8]. ReliSA [1] catalogue 168 pathologies de processus ; la plupart (réunions, management) n'apparaissent pas dans p4d. Les cinq motifs que nous ciblons (tableau I) proviennent de ce vocabulaire (Lone Wolf, Bystander Apathy, Warm Bodies, etc.) et sont ici opérationnalisés sur des événements de gestion de versions—comme **proxies** des fiches ReliSA les plus proches.

Observabilité. Prometheus définit quatre familles d'instruments : Counter, Gauge, Histogram et Summary [2] ; OpenMetrics standardise leur exposition [3]. Peu de travaux reproduisent ces types hors ligne sur des archives p4d complètes.

Traitement d'événements complexes. Les moteurs CEP évaluent des requêtes sur des flux d'événements non bornés [4] : ils maintiennent un état sur un flux ordonné, contrairement aux compteurs, et permettent par exemple edit→revert ou le cycle de checkout. Nous utilisons BEEPBEEP 3 [5], une bibliothèque Java où un calcul est un pipeline de processeurs composables. La même abstraction exprime à la fois les métriques agrégées de la phase 1 et les requêtes à base d'automates de la phase 2 ; le traitement événement par événement est essentiel à l'échelle de plusieurs gigaoctets.

III. MÉTHODOLOGIE

A. Modèle d'événements et antipatterns ciblés

Nous modélisons le journal comme un flux d'événements. Un événement est un tuple $e = (t, u, f, a)$ où t est un horodatage, u un utilisateur, f un chemin de fichier et a une action (edit, submit, sync, revert, etc.). Pour u , on note $E(u)$ et $S(u)$ le nombre d'edit et de submit. Les cinq antipatterns sont :

- 1) *Procrastination collective* : fenêtre w (heures avant échéance) où le taux de soumission dépasse la référence d'un facteur ρ (propriété agrégée).
- 2) *Warm Bodies* : $E(u) + S(u) < \theta_{wb}$ sur le projet (membre nominal mais inactif).
- 3) *Lone Wolf* : $E(u) / \max(S(u), 1) \geq \tau$ avec $S(u)$ faible (travail local, peu partagé).
- 4) *Bystander Apathy* : edit de f par u sans submit ultérieur de f par u avant fin de flux.
- 5) *Modification-puis-annulation* : edit de f par u puis revert sans submit intermédiaire.

Les motifs 1–3 sont des propriétés de comptage (phase 1). Les motifs 4–5 sont sensibles à l'ordre : ils exigent de mémoriser, fichier par fichier, qu'un edit a eu lieu et de guetter un submit ou revert (automates finis, phase 2). Onze commandes `user-*` sont parsées. Le tableau I relie chaque détection aux fiches ReliSA les plus proches (*proxies*, pas couverture exhaustive).

B. Architecture du pipeline

Fig. 1 résume PERFORCELOG : `log.txt` (~1,7 Go) → LecteurLog (flux + déduplication) → BEEPBEEP 3.13 (phases 1 et 2) → JSON (métriques, alertes, export `perforce_events_v3`, 27 colonnes). LecteurLog consomme le journal ligne par ligne sans charger tout le fichier et assemble des **blocs d'événement** (commande `user-*` + completed + métriques --, liés par pid). La déduplication retire les blocs répétés ; l'ordre chronologique est conservé pour le CEP. Les deux phases sont des chaînes de processeurs BEEPBEEP.

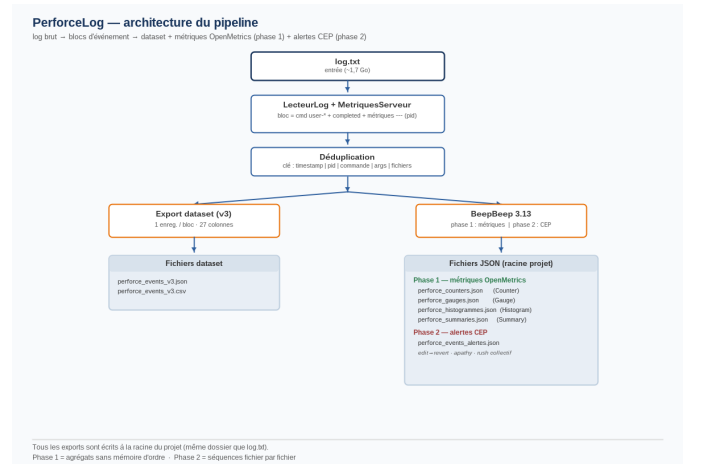


FIGURE 1. Pipeline en flux : parse → métriques → CEP.

C. Phase 1 : métriques d'observabilité

La phase 1 reproduit les quatre instruments OpenMetrics sur le flux : **Counter** (totaux monotones : événements, soumissions, modifications) ; **Gauge** (fichiers ouverts en modification, +1 sur edit, -1 sur submit/revert) ; **Histogram** (16

TABLE II
JEU DE DONNÉES ET RÉSUMÉ DE L'EXTRACTION.

Quantité	Valeur
Taille du journal brut	1,7 Go
Événements extraits	17 187
Événements uniques (après dédup.)	16 262
Lignes dupliquées retirées	925
Événements edit / submit	4 069 / 1 224
Temps de traitement	~14 s

buckets de durée) ; **Summary** (quantiles p50, p95, p99). Pipelines QueueSource→FilterOn→Cumulate ; automate editsOuverts. Ces instruments caractérisent volume et latence et soutiennent procrastination, Warm Bodies et Lone Wolf.

D. Phase 2 : traitement d'événements complexes

Chaque fichier est suivi par un automate de Moore : edit → ouvert ; submit → fermé ; revert depuis ouvert → alerte modification-puis-annulation ; ouvert en fin de flux → apathie. Rush collectif si ≥ 3 soumetteurs distincts/heure. La jauge phase 1 compte, par construction, les fichiers encore ouverts ; sa valeur terminale doit égaler le nombre d'alertes d'apathie. Les alertes sont listées dans un JSON dédié (fichier, utilisateur, horodatage, motif), distinct des métriques agrégées.

IV. RÉSULTATS

Nous avons exécuté PERFORCELOG sur un journal serveur Perforce de 1,7 Go issu d'un cours de développement de jeux multi-équipes (Unreal 5.6, NAD/NAND, 26–27 avril 2026). Le tableau II résume l'extraction ; le flux complet est traité en ~14 s sur un ordinateur portable courant, ce qui confirme que la conception en flux passe à l'échelle des journaux de plusieurs gigaoctets.

A. Activité dans le temps

Le taux de soumission présente la procrastination collective attendue (fig. 2, g.) : quasi nul la nuit, puis une montée jusqu'à un pic de **107** soumissions dans l'heure de 15 :00 (jour de remise), avec jusqu'à **60** utilisateurs distincts actifs en une seule heure. La jauge de fichiers ouverts monte et descend en conséquence (fig. 2, d.), culminant à **500** fichiers ouverts simultanément (**79** utilisateurs distincts au maximum).

B. Distribution des durées de commande

La fig. 3 montre l'histogramme des durées (16 232 observations, 16 buckets). Environ **89 %** des commandes s'exécutent en moins de 0,25 s ; une minorité — surtout des sync téléchargeant de gros assets Unreal — forme une longue traîne. Celle-ci tire la moyenne à 7,36 s alors que la médiane n'est que de 0,094 s, ce qui justifie que l'instrument Summary rapporte des quantiles plutôt qu'une simple moyenne (tableau III). La commande sync à elle seule atteint p99 = 1 102 s.

TABLE III
QUANTILES SUMMARY DES DURÉES DE COMMANDE (SECONDES).

Commande	p50	p95	p99
globale	0,094	1,144	85,1
edit	0,110	0,204	0,698
submit	0,125	0,250	0,821
sync	0,188	104,7	1 102

TABLE IV
RATIOS LONE WOLF LES PLUS EXTRÊMES (ANONYMISÉS).

Utilisateur	Modif.	Soum.	Ratio
U1	364	2	×182
U2	230	2	×115
U3	216	3	×72
U4	90	2	×45
U5	140	4	×35
U6	85	3	×28

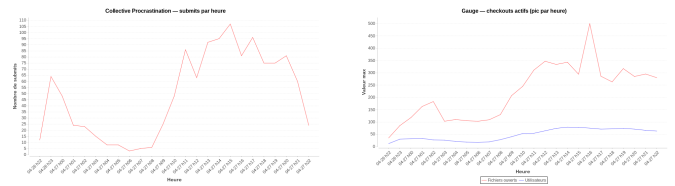


FIGURE 2. Activité sur 24 h autour de l'échéance. **G.** : soumissions par heure (pic à 107). **D.** : fichiers ouverts suivis par la jauge (pic à 500).

C. Antipatterns détectés

La phase CEP signale **217** séquences *modification-puis-annulation* et **250** fichiers en *Bystander Apathy* (fig. 4). Le rush collectif atteint 60 utilisateurs/heure (27/04, 13 h). Le motif Lone Wolf fait ressortir plusieurs contributeurs extrêmes ; le tableau IV liste les ratios edit/submit les plus élevés — le maximum atteint 364 modifications pour 2 soumissions (×182). De tels profils correspondent à un travail local important presque jamais partagé avec l'équipe.

D. Validation croisée des deux phases

Les deux phases sont volontairement redondantes sur une grandeur. La jauge phase 1 suit les fichiers ouverts par un solde +1/−1 et, en fin de flux, rapporte **250** fichiers encore ouverts. L'automate phase 2, par un mécanisme indépendant (séquence fichier par fichier), rapporte lui aussi exactement **250** alertes d'apathie. L'accord est exact : une erreur de comptage dans la jauge ou un défaut dans l'automate ferait diverger les deux nombres. En l'absence de vérité terrain, cette confirmation mutuelle constitue notre indice interne le plus solide que les deux phases sont correctes.

V. DISCUSSIONS ET LIMITES

Seuils et vérité terrain. Plusieurs motifs dépendent de seuils (tau, thet_wb, rho) dont le calibrage est propre au cours. Nous rapportons pour l'instant des preuves classées plutôt que des verdicts binaires, et nous n'avons pas encore validé un échantillon étiqueté d'alertes pour mesurer la précision ; c'est notre prochaine étape immédiate.

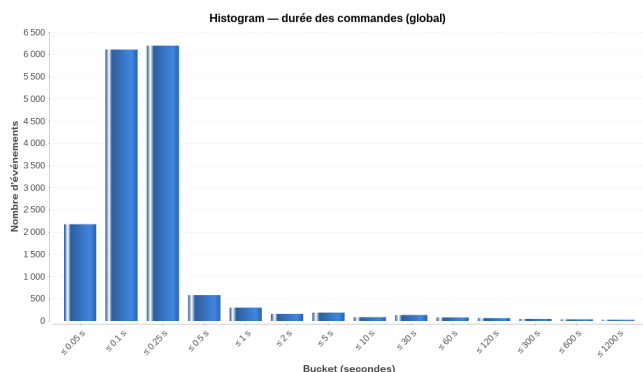


FIGURE 3. Histogramme des durées de commande (16 232 obs., 16 buckets). Comptes par borne sup. ($s \rightarrow n$) : 0,05→2 180 ; 0,1→6 114 ; 0,25→6 202 ; 0,5→584 ; 1→16 ; 2→9 ; 5→10 ; 10→5 ; 30→7 ; 60→4 ; 120→4 ; 300→3 ; 600→3 ; 1 200→3 ; $+\infty \rightarrow 3$.

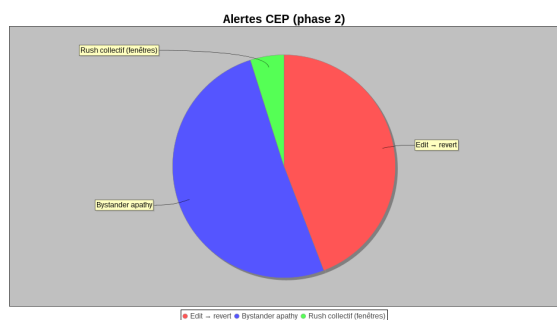


FIGURE 4. Alertes CEP phase 2 (edit→revert, apathie, rush).

Vue limitée au journal. L'approche ne voit que ce que le serveur enregistre. Un Lone Wolf peut se coordonner hors de la gestion de versions, et un fichier abandonné peut refléter une branche exploratoire légitime. L'outil fait émerger des candidats à l'interprétation humaine ; il n'attribue pas de blâme.

Sémantique de fin de flux. La validation croisée de la section 4.4 vaut par rapport à l'état final du journal. Un fichier réouvert puis refermé plus tard pourrait être compté différemment sous une définition de motif plus stricte ; nos deux phases concordent ici car elles partagent la même convention de fin de flux.

Travaux futurs. Nous prévoyons de (i) valider un échantillon aléatoire d'alertes contre le journal brut pour estimer la précision et le taux de faux positifs, (ii) anonymiser les noms d'utilisateurs pour une diffusion sûre, (iii) produire des tableaux de bord par équipe, et (iv) généraliser le pipeline au-delà de Perforce à d'autres journaux de gestion de versions au niveau du fichier.

VI. CONCLUSION

Nous avons présenté un outil en flux qui détecte cinq antipatterns de collaboration à partir des logs serveur Perforce en combinant des métriques de style Prometheus avec le traitement d'événements complexes de BeepBeep. Sur un journal de cours de 1,7 Go, il traite 16 262 événements en quelques

secondes et produit à la fois un portrait agrégé de l'activité de l'équipe et des alertes sensibles à l'ordre qu'un compteur sans mémoire ne peut produire. L'accord exact entre la jauge de fichiers ouverts et le compte d'apathie fondé sur l'automate illustre comment les deux phases se valident mutuellement. Le travail restant principal est la validation empirique de la précision des alertes et la diffusion d'une rétroaction anonymisée, par équipe, aux enseignants. PERFORCELOG enchaîne log brut Perforce, métriques OpenMetrics et alertes CEP. La phase 1 quantifie l'activité ; la phase 2 capture les séquences comportementales. Cohérence gauge/apathy (250/250).

Les pics de submits évoquent des remises de travail ; les syncs longues, un rattrapage d'assets binaires après absence ; les ratios edit/submit élevés, un travail local peu partagé.

Limites. Cinq comportements sur 168 ; log multi-projets ; antipatterns organisationnels invisibles dans le VCS ; validation qualitative des alertes à faire. Un seul dépôt universitaire.

Perspectives. Échantillon d'alertes vrai/faux positif ; hotspots de revert sur fichiers binaires ; table équipes officielle.

RÉFÉRENCES

- [1] ReliSA Software Process Antipatterns Catalogue. <https://github.com/ReliSA/Software-process-antipatterns-catalogue>
- [2] B. Brazil, *Prometheus : Up & Running*. Sebastopol, CA : O'Reilly Media, 2018.
- [3] OpenMetrics Authors, *OpenMetrics : An open standard for exposing metrics data*, 2020. [En ligne]. Disponible : <https://openmetrics.io>
- [4] D. C. Luckham, *The Power of Events : An Introduction to Complex Event Processing in Distributed Enterprise Systems*. Boston, MA : Addison-Wesley, 2002.
- [5] S. Halle, *Event Stream Processing with BeepBeep 3 : Log Crunching and Analysis with a Functional Approach*, 2018.
- [6] A. Hindle, D. M. German, and R. Holt, What do large commits tell us ? A taxonomical study of large commits, in *Proc. Int. Working Conf. Mining Software Repositories (MSR)*, 2008, pp. 99–108.
- [7] W. J. Brown, R. C. Malveau, H. W. McCormick, and T. J. Mowbray, *AntiPatterns : Refactoring Software, Architectures, and Projects in Crisis*. New York : Wiley, 1998.
- [8] D. A. Tamburri, F. Palomba, and R. Kazman, Exploring community smells in open-source : An automated approach, *IEEE Trans. Softw. Eng.*, vol. 47, no. 3, pp. 630–652, 2021.